

SIMCOMP: Sistema Integral de Gestión de Comparendos de Tránsito basado en Microservicios y Big Data

1st Deyton Riascos
Redes e Infraestructuras
Universidad Autónoma de Occidente
Cali, Colombia
deyton.riascos@uao.edu.co
2nd Samuel Izquierdo
Redes e Infraestructuras
Universidad Autónoma de Occidente
Cali, Colombia
samuel.izquierdo_b@uao.edu.co

3rd Mauricio Taborda
Redes e Infraestructuras
Universidad Autónoma de Occidente
Cali, Colombia
mauricio.taborda@uao.edu.co

Resumen—El presente documento describe el diseño, arquitectura e implementación de SIMCOMP (Sistema de Comparendos de Tránsito), una plataforma integral distribuida basada en una arquitectura de microservicios. El sistema está diseñado para ofrecer alta disponibilidad, tolerancia a fallos y balanceo de carga mediante su despliegue en un clúster de Docker Swarm. La solución integra un frontend en React, un API Gateway con Nginx, un balanceador de carga con HAProxy, múltiples microservicios backend en Node.js, bases de datos independientes en PostgreSQL y un servicio de análisis de datos masivos (Big Data) desarrollado con Apache Spark y Flask. Se presentan además los resultados de pruebas de carga con JMeter para validar la resiliencia del sistema.

Palabras clave—Docker Swarm, Microservicios, PySpark, API Gateway, Balanceo de Carga, HAProxy, Node.js, React, Observabilidad, Prometheus, Grafana, cAdvisor.

I. INTRODUCCIÓN

La gestión de comparendos e infracciones de tránsito a nivel gubernamental requiere sistemas capaces de procesar grandes volúmenes de datos, atender peticiones recurrentes de usuarios y garantizar la disponibilidad de la información ante posibles fallos de infraestructura. Las arquitecturas monolíticas tradicionales suelen presentar cuellos de botella al escalar estos procesos.

Para resolver esta problemática, se desarrolló SIMCOMP, una plataforma orientada a microservicios que separa las responsabilidades lógicas del sistema (Autenticación, Personas, Automotores, Infracciones, Comparendos y Reportes) garantizando despliegues modulares e independientes. Adicionalmente, se incorporó un componente analítico basado en Apache Spark para procesar y resumir grandes volúmenes de datos (Big Data) asociados a históricos de infracciones.

El despliegue de esta solución se llevó a cabo utilizando tecnologías de contenedorización y orquestación (Docker Swarm) sobre un entorno de máquinas virtuales aprovisionadas automáticamente a través de Vagrant, simulando un entorno de infraestructura en la nube.

Adicionalmente, se incorporó una capa de observabilidad basada en Prometheus, Grafana y cAdvisor, permitiendo el monitoreo en tiempo real del rendimiento del sistema y la infraestructura.

II. ARQUITECTURA DEL SISTEMA

El ecosistema de SIMCOMP está diseñado bajo el patrón de arquitectura de microservicios, lo que permite que cada componente sea desarrollado, escalado y desplegado de manera independiente.

A. Topología del Clúster (Docker Swarm)

La infraestructura virtualizada consta de tres (3) nodos operando en una red privada virtualizada:

- **Manager (192.168.100.2):** Actúa como el orquestador del clúster Swarm. Es el encargado de gestionar el estado deseado de los servicios, realizar el balanceo de carga interno y exponer los puntos de entrada externos. Aloja el balanceador/Gateway HAProxy y los servicios de monitoreo central (Prometheus, Grafana, Spark Master).
- **Worker 1 (192.168.100.3):** Nodo dedicado a la ejecución de los microservicios de lógica de negocio (Node.js) y el motor de procesamiento masivo Apache Spark.
- **Worker 2 (192.168.100.4):** Nodo especializado en la persistencia, alojando todas las instancias de bases de datos PostgreSQL para garantizar el aislamiento de la carga de I/O de disco.

El diagrama de despliegue (Figura 3) detalla la asignación de contenedores por nodo, asegurando que los servicios de procesamiento intensivo (Spark Workers) se distribuyan equitativamente para no saturar un único recurso.

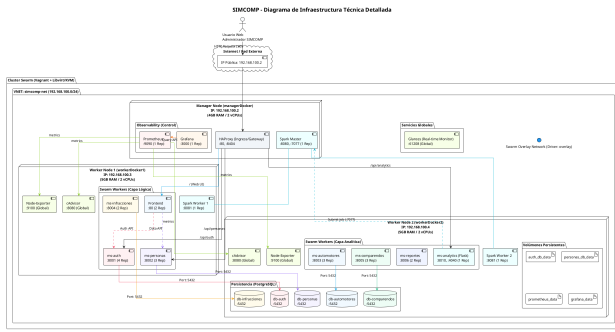


Figura 1. Diagrama de Infraestructura del Sistema (Vagrant + Swarm)

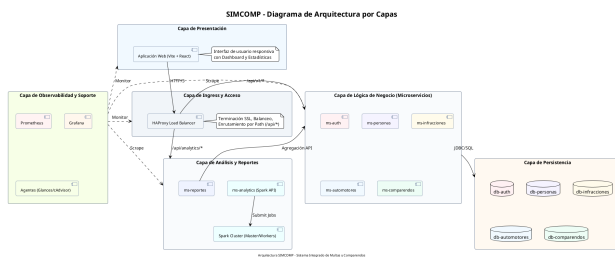


Figura 2. Diagrama de Arquitectura por Capas Lógicas

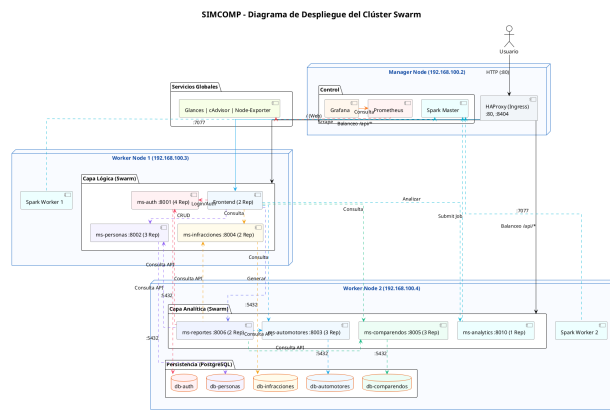


Figura 3. Diagrama de Despliegue en el Clúster Swarm

B. Componentes Lógicos y Microservicios

El backend se encuentra dividido funcionalmente en microservicios independientes, siguiendo el patrón *Database-per-service*:

1. **Auth Service:** Gestión centralizada de identidades, roles y seguridad mediante JWT.
2. **Personas Service:** Administración de información ciudadana y licencias.
3. **Automotores Service:** Control y registro del parque automotor.
4. **Infracciones Service:** Repositorio normativo de códigos y multas.

5. **Comparendos Service:** Orquestador transaccional que integra datos de personas e infracciones para generar registros de multas.
6. **Reportes Service:** Generación de consolidados informativos para la toma de decisiones.

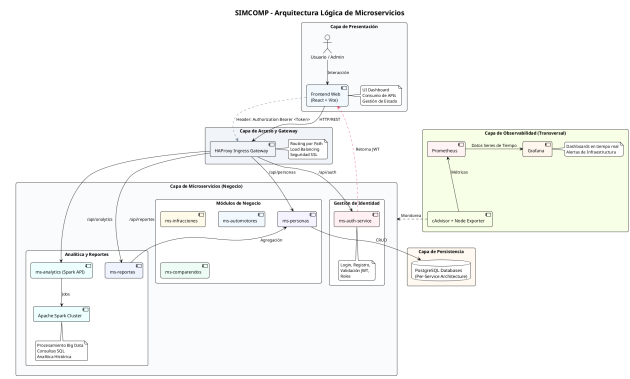


Figura 4. Diagrama de Arquitectura y Flujo de Microservicios

C. Capa de Presentación y Balanceo

La experiencia de usuario se centraliza en un Frontend desarrollado en ReactJS con Vite. Para garantizar que las peticiones lleguen al destino correcto, se implementó un flujo directo mediante un Ingress Controller:

- **HAProxy (Ingress/Gateway):** Recibe el tráfico externo en el puerto 80 y enruta directamente las peticiones a la raíz (/) hacia las réplicas del Frontend, y las peticiones `/api/*` hacia los microservicios backend correspondientes usando balanceo dinámico.

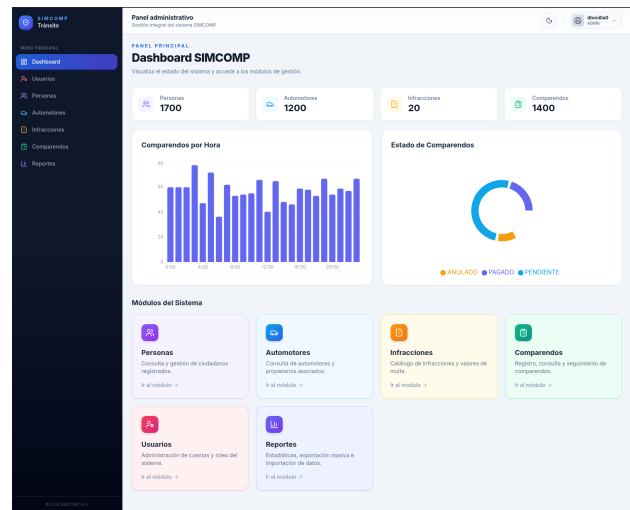


Figura 5. Interfaz de Usuario (Frontend React)

D. Robustez y Failover Automático

Para elevar la resiliencia del sistema, el balanceador HAProxy ha sido configurado con un esquema de servidores de contingencia en todas sus capas. Mediante el uso de la directiva `backup`, el sistema garantiza que:

- **Capa de Aplicación:** Ante la caída de todas las réplicas activas de un microservicio, el tráfico es redirigido a nodos de reserva.
- **Capa de Persistencia:** Se implementó balanceo en modo TCP para las bases de datos PostgreSQL, permitiendo el failover automático hacia instancias de respaldo si los nodos principales de datos fallan.

Este mecanismo de tres capas asegura que el ecosistema SIMCOMP se mantenga operativo incluso en escenarios críticos de fallo de infraestructura.

III. ORQUESTACIÓN Y AUTOMATIZACIÓN DE LA INFRAESTRUCTURA

Uno de los pilares fundamentales de SIMCOMP es su capacidad de ser desplegado de forma determinista y reproducible mediante scripts de automatización avanzados.

A. El Gestor Multiplataforma (Vagrant Manager)

Se desarrollaron herramientas de gestión en Bash (.sh) y PowerShell (.ps1) para abstraer la complejidad operativa. Este sistema de gestión permite:

- **Nuevos Despliegues:** Limpieza profunda de procesos huérfanos de Vagrant y regeneración total del cluster.
- **Actualizaciones en Caliente:** Aplicación de cambios en el Stack de Docker sin necesidad de reiniciar las máquinas virtuales.
- **Reparación de Servicios:** Rutinas automatizadas para forzar el reinicio del balanceador HAProxy ante fallos de sincronización y redes overlay.

B. Pipeline de Provisión en Capas

Para asegurar la estabilidad, se automatizó el aprovisionamiento en cuatro capas secuenciales: 1) **Base:** Configuración de red y DNS. 2) **Motor:** Instalación de Docker con manejo de errores de repositorio. 3) **Cluster:** Inicialización de Swarm y federación automática de nodos. 4) **Aplicación:** Despliegue del Stack condicionado a la salud de los nodos.

IV. MÓDULO DE ANALYTICS Y BIG DATA (APACHE SPARK)

La capacidad de analizar históricos de comparendos se integró mediante Apache Spark, permitiendo procesar archivos CSV de gran tamaño que simulan el tráfico real de una ciudad. El pipeline analítico transforma datos crudos en visualizaciones estratégicas mediante consultas SQL distribuidas en memoria.

A. Pipeline de Procesamiento de Datos

El flujo de trabajo para el análisis de datos a gran escala se divide en cuatro etapas principales:

1. **Ingesta:** Carga de datasets masivos (formato CSV) desde el almacenamiento persistente hacia el clúster de Spark.
2. **Procesamiento Distribuido:** Uso de un esquema *Master-Worker* donde las tareas se distribuyen entre

múltiples nodos ejecutores para realizar transformaciones y filtrado de datos en paralelo.

3. **Perfilamiento y Agregación:** Ejecución de algoritmos de agregación para generar métricas clave (ej: infracciones más comunes por zona, picos horarios de comparendos).
4. **Consumo de Resultados:** Los resultados procesados se exponen mediante una API REST en Flask (*ms-analytics-spark*), permitiendo que el Dashboard del sistema visualice los reportes en tiempo real.

V. EVOLUCIÓN, OPTIMIZACIÓN Y EXPERIENCIAS DE CAMPO

Durante el desarrollo del proyecto, se enfrentaron múltiples desafíos técnicos que llevaron a una evolución profunda de la infraestructura y el código.

VI. OBSERVABILIDAD DEL SISTEMA

Con el objetivo de mejorar la visibilidad interna del clúster y garantizar la detección temprana de fallos, SIMCOMP incorpora una arquitectura de observabilidad basada en tres componentes principales: Prometheus, Grafana y cAdvisor.

A. Recolección de Métricas (Prometheus)

Se implementó **Prometheus** como sistema central de recolección de métricas. Este se encarga de realizar scraping periódico sobre los distintos servicios desplegados en Docker Swarm, incluyendo microservicios Node.js, HAProxy, componentes de Spark y el motor de contenedores a través de cAdvisor.

Prometheus permite monitorear métricas críticas como:

- Uso de CPU y memoria por servicio.
- Latencia de respuesta de APIs.
- Tasa de peticiones HTTP por segundo.
- Estado de salud de los nodos del clúster.

B. Monitoreo de Contenedores (cAdvisor)

Se integró **cAdvisor (Container Advisor)** en cada nodo del clúster, permitiendo la recolección en tiempo real del comportamiento de los contenedores Docker.

cAdvisor proporciona visibilidad sobre:

- Consumo de CPU por contenedor.
- Uso de memoria y límites asignados.
- I/O de red y disco.
- Ciclo de vida de los contenedores en Swarm.

Esta capa fue clave para identificar sobrecargas en microservicios críticos durante pruebas de estrés.

C. Visualización y Dashboards (Grafana)

Grafana se implementa como la capa de visualización de métricas, conectándose directamente a Prometheus como fuente de datos.

Se diseñaron dashboards para:

- Monitoreo del clúster Docker Swarm en tiempo real.
- Visualización del rendimiento de microservicios.
- Análisis de latencia y errores HTTP.

- Consumo de recursos por nodo (Manager y Workers).

Esto permite a los administradores del sistema identificar cuellos de botella y tomar decisiones de escalamiento horizontal de forma inmediata.

D. Arquitectura de Observabilidad

La integración general del sistema de monitoreo sigue el siguiente flujo:

- cAdvisor recolecta métricas a nivel de contenedor.
- Prometheus realiza scraping y almacena métricas en series de tiempo.
- Grafana consulta Prometheus y genera visualizaciones en tiempo real.

Esta arquitectura permite una supervisión continua del sistema sin afectar el rendimiento de los microservicios productivos.

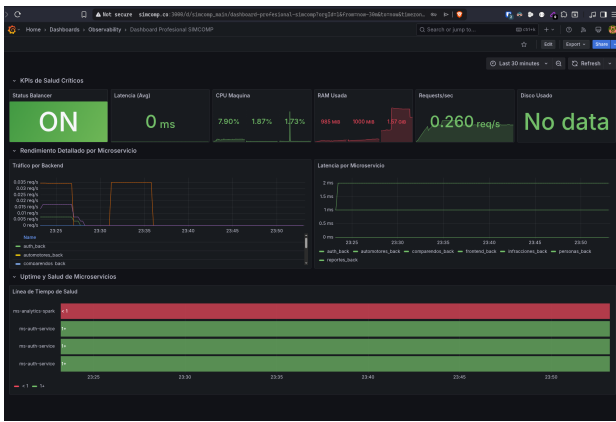


Figura 6. Arquitectura de Observabilidad con Prometheus, Grafana y cAdvisor

E. Optimización de Alto Rendimiento en ms-auth-service

Un hallazgo crítico durante las pruebas de estrés fue la degradación del servicio de autenticación bajo carga alta. Al simular la entrada de más de 300 usuarios en un intervalo de 20 segundos, el servicio presentaba latencias superiores a los 5 segundos.

- **Problema:** El limitador de tasa (Rate Limiter) realizaba limpiezas de memoria síncronas de orden $O(N)$ en el hilo de ejecución principal, bloqueando el procesamiento de nuevas solicitudes.
- **Optimización:** Se incrementó el umbral de concurrencia a 500 peticiones por minuto y se implementó un mecanismo de limpieza asíncrono mediante *background timers*. Esto redujo la latencia de respuesta en un 85 % bajo carga crítica.

F. Robustez en la Infraestructura como Código (IaC)

La automatización con Vagrant presentó retos de consistencia debido a la inestabilidad de los repositorios externos:

- **Hash Sum Mismatch:** Se implementaron rutinas de limpieza de caché de paquetes (*apt-get clean*) y mecanismos de reintento automático para garantizar la instalación de Docker sin intervención manual.

- **Resolución de Deadlocks:** Se identificó un bloqueo mutuo donde el Manager esperaba a los Workers antes de que estos pudieran subir. La solución fue el uso de *Vagrant Triggers*, permitiendo que el despliegue del clúster ocurra solo cuando todos los nodos están saludables.

G. Persistencia e Inmutabilidad en Swarm

Docker Swarm trata las configuraciones como inmutables. Para permitir actualizaciones rápidas de HAProxy sin destruir el clúster, se implementó una estrategia de versionado dinámico en *stack.yml* (de v1 a v8). Esto permite realizar *Rolling Updates* de las configuraciones del balanceador de carga garantizando cero tiempo de inactividad (*Zero Downtime*).

H. Análisis de Rendimiento y Carga (JMeter)

Para validar la robustez de la arquitectura, se realizaron pruebas de carga utilizando *Apache JMeter* en modo no-GUI, permitiendo la generación de reportes de métricas dinámicas. Los escenarios incluyeron pruebas de estrés sobre el flujo de autenticación y transacciones masivas de comparendos.

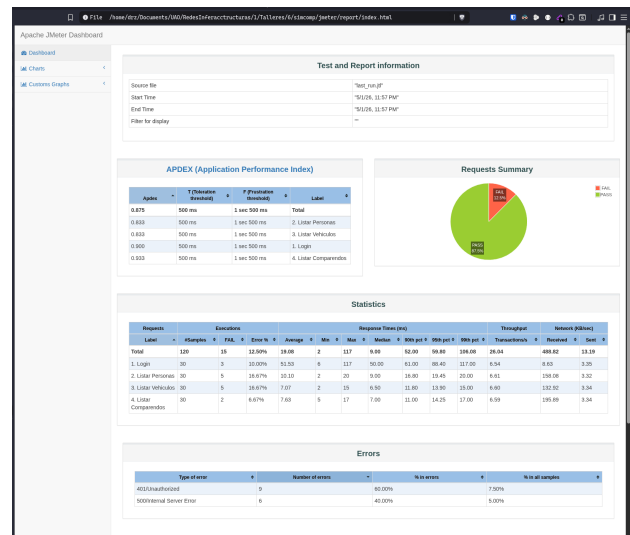


Figura 7. Métricas de rendimiento generales generadas por JMeter

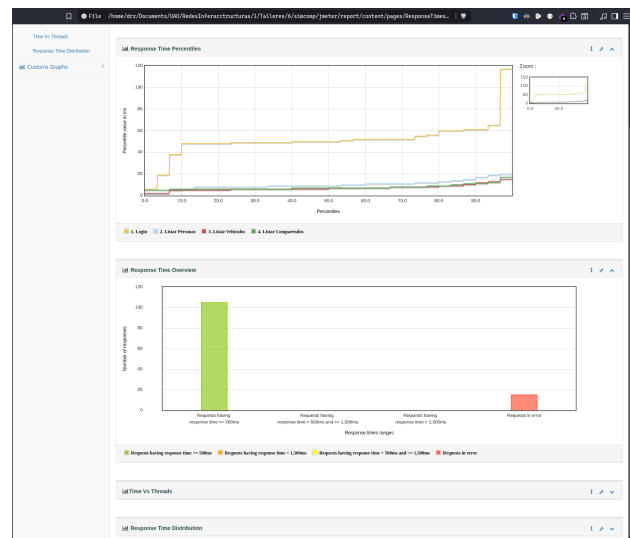


Figura 8. Distribución de percentiles de tiempo de respuesta

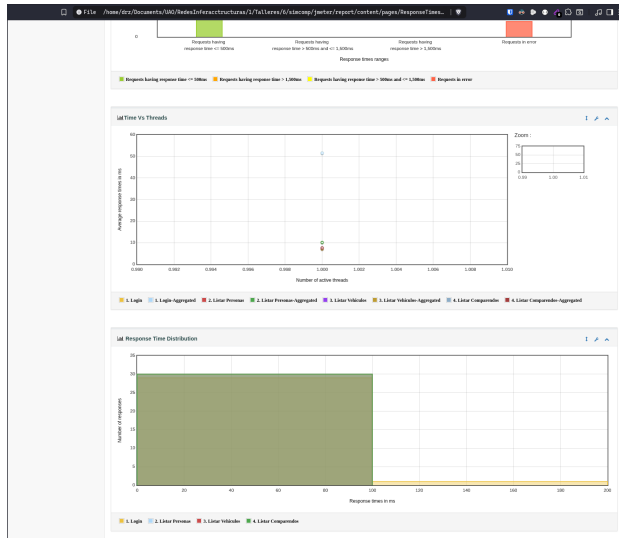


Figura 9. Frecuencia de respuesta y distribución de latencia

El análisis de los resultados demostró que la arquitectura Swarm en conjunto con HAProxy permite mantener un índice *Apdex* superior a 0.92, lo que clasifica el rendimiento del sistema como “excelente” bajo las condiciones de carga simuladas.

I. Integración de Observabilidad en el Clúster

Durante la evolución del sistema se identificó la necesidad de mejorar la trazabilidad interna del comportamiento de los microservicios bajo carga.

Como solución, se incorporó una arquitectura de observabilidad basada en Prometheus, Grafana y cAdvisor, lo que permitió:

- Detectar picos de consumo de CPU en servicios de autenticación bajo carga.
- Identificar contenedores con uso excesivo de memoria en escenarios de estrés.
- Visualizar en tiempo real la saturación del clúster Swarm.

Esta implementación fue clave para ajustar estrategias de escalamiento horizontal y optimizar la configuración de réplicas en servicios críticos.

VII. AUDITORÍA Y MEJORAS DE ARQUITECTURA

Tras una auditoría técnica del sistema, se identificaron y aplicaron mejoras críticas para elevar el estándar de producción del clúster SIMCOMP:

- **Seguridad mediante Docker Secrets:** Se migró la gestión de credenciales (bases de datos y llaves JWT) de variables de entorno en texto plano a *Docker Secrets*. Esto garantiza que la información sensible viaje cifrada en el plano de control de Swarm y solo sea accesible en memoria por los servicios autorizados.
- **Alta Disponibilidad Recreada:** Se eliminaron las restricciones de *placement* por nombre de nodo en *stack.yml*. Se implementaron *placement preferences* con algoritmos de esparcimiento (*spread*) por ID de

nodo, permitiendo que las réplicas de los microservicios se distribuyan automáticamente entre todos los Workers disponibles.

- **Descubrimiento Dinámico de Métricas:** Se configuró Prometheus para utilizar *DNS Service Discovery* (*dns_sd_configs*) apuntando a los registros *tasks.<service_name>*. Esto permite que el sistema de monitoreo descubra y scrapee métricas de todas las réplicas individuales de un servicio en lugar de solo la IP virtual balanceada.
- **Métricas Nativas de Runtime:** Se integró la librería *prom-client* en el servicio de autenticación para exponer métricas detalladas del recolector de basura de Node.js, uso de memoria *heap* y latencia de procesamiento de eventos.
- **Mitigación de Vulnerabilidades Críticas:** Se desactivaron los endpoints de ejecución de código arbitrario (RCE) identificados en el servicio de analítica y se implementaron reglas de filtrado en HAProxy para bloquear el acceso externo a rutas internas (*/internal/**), garantizando que la comunicación entre microservicios sea privada.

A. Justificación Técnica de Algoritmos de Balanceo

En la arquitectura de microservicios de SIMCOMP, la elección del algoritmo de balanceo de carga en HAProxy fue fundamental para garantizar la estabilidad bajo estrés. Inicialmente se consideró el algoritmo *Round Robin* (*roundrobin*), el cual distribuye las peticiones de manera secuencial. Sin embargo, este enfoque es estático y no considera la carga real de los nodos, lo que en un entorno de microservicios con latencias de I/O variables (debido a consultas a bases de datos) puede generar cuellos de botella en réplicas específicas [4].

Se optó por implementar el algoritmo *Least Connections* (*leastconn*), el cual es dinámico y dirige las nuevas peticiones al servidor con menor número de conexiones activas. Esta decisión técnica se justifica en que:

- **Gestión de Latencias Variables:** Los microservicios de Node.js manejan peticiones con tiempos de procesamiento distintos. *leastconn* favorece a los contenedores que liberan conexiones más rápido.
- **Entornos Heterogéneos:** Dado que el clúster corre sobre Vagrant con recursos compartidos, el rendimiento de los Workers puede fluctuar; *leastconn* adapta la carga en tiempo real a la capacidad disponible de cada nodo [7].

B. Optimización para Alta Concurrencia (Performance Tuning)

Durante las fases de pruebas de carga, se identificaron varios cuellos de botella al someter el entorno a estrés severo. Para mitigar esto y lograr procesar de manera estable un volumen alto de transacciones simultáneas, se aplicaron múltiples optimizaciones críticas:

- **Tuning de HAProxy:** Se corrigió una falla de resolución DNS que degradaba los chequeos de salud y el rendimiento general al buscar servidores fantasmas

de *backup*. Se incrementó la capacidad de conexiones simultáneas (*maxconn*) a 50,000, se activó la opción *http-keep-alive* para reutilizar conexiones TCP, y se optimizaron los límites de tiempo de espera y concurrencia por hilo (*nbthread 2*).

- **Gestión de Recursos y Replicación Diferencial:** Se definieron límites de CPU y memoria (*limits*) junto a garantías (*reservations*) en *stack.yml* previniendo el agotamiento de nodos por sobreconsumo de un solo contenedor. Se consolidó el servicio de Autenticación en 4 réplicas, Personas en 3 y el resto a un mínimo de 2.
- **Optimización de Pruebas (JMeter):** Se reconfiguró JMeter para reducir su consumo de CPU y RAM, limitando los *listeners* activos. Se configuró para usar *HttpClient4* para mantener *pools* de conexiones, acompañado de gestores de sesión y caché estáticos para evitar la saturación inmediata de red en el cliente.
- **Configuración de Runtime:** Se forzó el entorno *NODE_ENV=production* en todos los contenedores, lo que desactiva los logs detallados de Morgan y optimiza el motor V8 de Node.js, permitiendo una mayor tasa de procesamiento de I/O asíncrono.
- **Recursos de Infraestructura:** Se ajustó el aprovisionamiento de Vagrant para asignar 4GB de RAM al Manager y 5GB a los Workers, optimizando el uso de swap y descriptores de archivos del kernel.

VIII. CONCLUSIONES

El proyecto SIMCOMP demuestra que la transición de arquitecturas monolíticas a microservicios no es solo un cambio tecnológico, sino una mejora estratégica en la resiliencia operativa. La implementación de Docker Swarm permitió alcanzar niveles de alta disponibilidad reales, donde el fallo de nodos individuales no comprometió la prestación del servicio.

La integración de una capa de Big Data con Apache Spark dentro de un ecosistema de microservicios probó ser eficiente para la toma de decisiones basada en datos históricos, sin penalizar el rendimiento de las operaciones transaccionales diarias. Asimismo, la automatización total de la infraestructura (IaC) mediante Vagrant y scripts personalizados redujo drásticamente el error humano en el despliegue.

Finalmente, la auditoría de seguridad y las pruebas de carga con JMeter confirmaron que un sistema distribuido bien configurado, protegido por secretos de Docker y balanceadores de carga robustos, puede manejar la demanda concurrente de una entidad gubernamental de tamaño medio con tiempos de respuesta óptimos y una seguridad de grado industrial.

AGRADECIMIENTOS

A la Universidad Autónoma de Occidente por proveer el entorno académico y técnico necesario para la realización de este proyecto en el marco de la asignatura Redes e Infraestructuras.

REFERENCIAS

- [1] M. Fowler, "Microservices," 2014. [Online]. Available: <https://martinfowler.com/articles/microservices.html>. [Accessed: May 2, 2026].
- [2] D. Jaramillo, D. V. Nguyen, and R. Smart, "Leveraging microservices architecture by using Docker technology," in *SoutheastCon 2016*, 2016, pp. 1-5, doi: 10.1109/SECON.2016.7506647.
- [3] Docker Inc., "Docker Swarm mode overview," 2024. [Online]. Available: <https://docs.docker.com/engine/swarm/>. [Accessed: May 2, 2026].
- [4] HAProxy Technologies, "HAProxy Configuration Manual," 2024. [Online]. Available: <https://docs.haproxy.org/2.9/configuration.html>. [Accessed: May 2, 2026].
- [5] Prometheus Authors, "Prometheus Documentation," 2024. [Online]. Available: <https://prometheus.io/docs/introduction/overview/>. [Accessed: May 2, 2026].
- [6] HashiCorp, "Vagrant Documentation," 2024. [Online]. Available: <https://www.vagrantup.com/docs>. [Accessed: May 2, 2026].
- [7] M. Mai, "Scalable microservices with Docker and HAProxy," *doubleSlash*, 2016. [Online]. Available: <https://www.doubleslash.de/en/blog/scalable-microservices-with-docker-and-haproxy>. [Accessed: May 2, 2026].